

Scenario 2 Submission for the FJSSP-W Competition: A Surrogate-Assisted Genetic Algorithm for Robust Scheduling

Johann Bartels
Leibniz Universität Hannover
johann.ulrich.bartels@stud.uni-hannover.de

Abstract

This report describes my submission for Scenario 2 of the FJSSP-W Competition. The solver targets robust scheduling under worker-dependent processing-time uncertainty and is based on a genetic algorithm for the Flexible Job Shop Scheduling Problem with Worker Flexibility (FJSSP-W). The original C# implementation was translated to Python and integrated into the competition repository. Candidate solutions encode an operation sequence, machine assignments, and worker assignments. The objective used during optimization is the simulated robust makespan under the fixed uncertainty parameters of Scenario 2. To reduce the cost of repeated stochastic evaluations, the algorithm combines real simulation calls with a quantile-random-forest surrogate model and a restricted local search around promising candidates. The final submitted solution of each run is reevaluated with 50 stochastic simulations and exported in the required Scenario-2 CSV format.

1 Implementation Basis

The foundation of the solver is a robust genetic algorithm for FJSSP-W. The original implementation was taken from the repository by Hutter, Steinberger, and Hellwig [1]. This repository contains a C# implementation of a genetic algorithm that is designed to produce feasible FJSSP-W schedules. For this submission, the relevant parts of the solver were translated from C# to Python and adapted to the official competition code base. The basic representation of an individual was kept. Each candidate consists of an operation sequence, machine assignments, and worker assignments. The construction and mutation operators are restricted to valid machine-worker combinations with positive processing times. Thus, the search is designed to generate feasible candidate schedules. Additional validation checks are applied after decoding to reject schedules that violate temporal consistency, job precedence, machine capacity, worker capacity, or acyclicity.

Several parts of the original evaluation pipeline were modified for the competition environment. In the original solver, functions such as `evaluate()` compute the fitness of an individual by simulating the schedule and tracking processing times. In this Python version, the deterministic part of the evaluation was replaced by the official `translate()` and `makespan()` functions provided in the competition repository. `translate()` decodes the operation sequence, machine assignments, and worker assignments into start times and assignments in the official operation order. `makespan()` is then used to compute the deterministic makespan of the decoded schedule.

For Scenario 2, the deterministic evaluation was extended by stochastic simulation. The resulting robust makespan is used as the individual's fitness. In the implementation, the `makespan` entry of `ind.fitness` is set to the robust makespan, and

`robust_makespan_stdev` and `R` are stored as additional fitness values.

2 Solution Representation

Each individual consists of

$$x = (\pi, m, w),$$

where π is a permutation-like operation sequence represented by job IDs, m is the machine assignment vector, and w is the worker assignment vector. The sequence determines the decoding order. Machine and worker assignments are stored in the fixed benchmark operation order. During initialization and mutation, only valid machine-worker combinations with positive processing times are used.

A candidate is decoded using the official benchmark representation. After decoding, the schedule is checked for job precedence, resource conflicts and acyclicity. Invalid candidates receive infinite fitness and are not used as final solutions.

3 Genetic Algorithm

Starting from the translated robust GA, the algorithm was adapted to the competition's Python evaluation pipeline and extended for Scenario 2. The GA maintains a population of feasible FJSSP-W individuals. Selection is performed by tournament selection. Recombination combines job subsequences from two parents and inherits machine-worker assignments from either parent. Mutation can affect three components: the operation sequence, the machine assignment, or the worker assignment. Elitism preserves the best individuals of each generation.

The base configuration used a population size of 200 and 1000 offspring per generation. The mutation probability is initialized as

$$p_m = \frac{1}{2n},$$

where n is the number of operations. The algorithm stops when the function evaluation budget or the optional wall-clock limit is reached.

4 Robust Evaluation

During search, candidates are evaluated by stochastic simulation using the Scenario-2 uncertainty parameters. The internal evaluation uses 10 simulations per candidate. The best solution found in a run is then reevaluated using 50 final simulations. The final robust makespan from this reevaluation is written to the official submission file as the fitness value.

The same uncertainty parameters are used for optimization, final evaluation, and CSV export. The uncertainty parameters

are generated once before the experiment and then reused unchanged for all corresponding runs.

5 Surrogate Model and Local Search

Because stochastic simulation is expensive, a surrogate model is used to reduce the number of full real evaluations. The model is a quantile random forest trained on previously simulated candidates. Features are extracted from the decoded schedule, including deterministic makespan, assignment information, timing features, and uncertainty-related features.

After a warm-up phase with real evaluations, the surrogate predicts robust fitness values for many generated offspring. A small subset is selected for real simulation: the best predicted candidates, uncertain candidates, and a small random fraction. This keeps exploration while still reducing the number of costly simulation calls.

A local search is periodically applied to promising real-evaluated individuals. It generates neighbors by changing workers, changing machine-worker pairs, swapping sequence positions, and shifting sequence positions. The surrogate filters these neighbors, and only selected candidates are evaluated by real simulation.

6 Algorithm Outline

Algorithm 1 Surrogate-assisted GA for Scenario 2

```

1: Load benchmark instance and fixed uncertainty parameters
2: Initialize population with feasible random individuals
3: Evaluate initial population by stochastic simulation
4: Train surrogate after sufficient real samples
5: while evaluation budget not exhausted do
6:   Select parents by tournament selection
7:   Create offspring by crossover
8:   Mutate sequence, machine, and worker components
9:   Decode and validate offspring
10:  if surrogate is ready then
11:    Predict robust fitness for offspring
12:    Select top, uncertain, and random candidates
13:    Evaluate selected candidates by real simulation
14:  else
15:    Evaluate offspring by real simulation
16:  end if
17:  Update surrogate with new real samples
18:  Apply elitist replacement
19:  if local search interval reached then
20:    Generate local neighbors of best individuals
21:    Filter neighbors using surrogate predictions
22:    Evaluate selected neighbors by real simulation
23:    Replace origin if improved
24:  end if
25: end while
26: Reevaluate best solution with 50 final simulations
27: Export official Scenario-2 CSV row

```

7 Hardware.

The experiments were executed on the Leibniz University Hannover Slurm cluster. The completed Scenario-2 job ar-

ray was distributed over heterogeneous compute nodes in the `mpp.share`, `taurus`, and `amo` partitions. Each array task allocated 26 CPU cores and requested 65,000 MB of memory. The used node classes were: `mpp.share` nodes with 128 CPUs and 500 GB RAM per node (AMD, AVX512, NDR200, year 2025), `amo` nodes with 40 CPUs and 180 GB RAM per node (Intel, AVX512, HDR100, year 2020), and `taurus` nodes with 32 CPUs and 120 GB RAM per node (Intel, AVX512, FDR10, year 2019). The software environment was Rocky Linux 9.4 with Python 3.9.18. Reported runtimes are Slurm wall-clock runtimes on heterogeneous cluster hardware.

8 Submission Contents

The submission archive contains the solver source code, the fixed Scenario-2 uncertainty configuration, the official result CSV file, additional metadata CSV files, validation scripts, and this report. The official CSV follows the required Scenario-2 format:

```

Instance; Fitness; FunctionEvaluations; StartTime;
MachineAssignments; WorkerAssignments; Uncertainty;

```

A validation script checks the number of rows, column order, JSON formatting of vectors, function-evaluation limits, uncertainty-parameter consistency, and basic schedule feasibility.

9 Discussion

The main strength of the approach is that it directly optimizes the stochastic Scenario-2 objective instead of optimizing only deterministic makespan. The surrogate model makes the expensive robust evaluation more tractable, and the local search provides targeted improvement around promising individuals.

The main limitation is the high computational cost. Since robust evaluation is simulation-based, the number of fully simulated candidates is limited. In addition, the experiments were run on heterogeneous cluster nodes, so wall-clock runtime values are not directly comparable to controlled single-node benchmark times. Further improvements would include stronger critical-path-based local search, adaptive simulation allocation, and instance-specific parameter tuning.

References

- [1] D. Hutter, M. Steinberger, and M. Hellwig. Robust genetic algorithm for the FJSSP-W. GitHub repository, accessed 2026. Available at: <https://github.com/jrc-rodec/FJSSPW-GA>.